How "Exit Traps" Can Make Your Bash Scripts Way More Robust And Reliable

There is a simple, useful idiom to make your bash scripts more robust - ensuring they always perform necessary cleanup operations, even when something unexpected goes wrong. The secret sauce is a pseudo-signal provided by bash, called EXIT, that you can trap (http://www.gnu.org/software/bash/manual/bashref.html#index-trap); commands or functions trapped on it will execute when the script exits for any reason. Let's see how this works.

The basic code structure is like this:

```
#!/bin/bash
function finish {
  # Your cleanup code here
}
trap finish EXIT
```

You place any code that you want to be certain to run in this "finish" function. A good common example: creating a temporary scratch directory, then deleting it after.

```
#!/bin/bash
scratch=$(mktemp -d -t tmp.XXXXXXXXXX)
function finish {
  rm -rf "$scratch"
}
trap finish EXIT
```

You can then download, generate, slice and dice intermediate or temporary files to the `$scratch` directory to your heart's content. [1]

```
# Download every linux kernel ever.... FOR SCIENCE!
for major in {1..4}; do
  for minor in {0..99}; do
    for patchlevel in {0..99}; do
      tarball="linux-${major}-${minor}-${patchlevel}.tar.bz2"
      curl -q "http://kernel.org/path/to/$tarball" -o "$scratc
h/$tarball" || true
      if [ -f "$scratch/$tarball" ]; then
        tar jxf "$scratch/$tarball"
      fi
    done
  done
done
# magically merge them into some frankenstein kernel ...
# That done, copy it to a destination
cp "$scratch/frankenstein-linux.tar.bz2" "$1"
# Here at script end, the scratch directory is erased automatic
ally
```

Compare this to how you'd remove the scratch directory without the trap:

```
#!/bin/bash
# DON'T DO THIS!
scratch=$(mktemp -d -t tmp.XXXXXXXXXX)

# Insert dozens or hundreds of lines of code here...

# All done, now remove the directory before we exit
rm -rf "$scratch"
```

What's wrong with this? Plenty:

- If some error causes the script to exit prematurely, the scratch directory and its contents don't get deleted. This is a resource leak, and may have security implications too.
- If the script is designed to exit before the end, you must manually copy 'n paste the rm command at each exit point.
- There are maintainability problems as well. If you later add a new in-script exit, it's easy to forget to include the removal - potentially creating mysterious heisenleaks.

Keeping Services Up, No Matter What

Another scenario: Imagine you are automating some system administration task, requiring you to temporarily stop a server... and you want to be dead certain it starts again at the end, even if there is some runtime error. Then the pattern is:

```
function finish {
    # re-start service
    sudo /etc/init.d/something start
}
trap finish EXIT
sudo /etc/init.d/something stop
# Do the work...

# Allow the script to end and the trapped finish function to st
art the
# daemon back up.
```

A concrete example: suppose you have MongoDB running on an Ubuntu server, and want a cronned script to temporarily stop the process for some regular maintenance task. The way to handle it is:

```
function finish {
    # re-start service
    sudo service mongdb start
}
trap finish EXIT
# Stop the mongod instance
sudo service mongdb stop
# (If mongod is configured to fork, e.g. as part of a replica s
et, you
# may instead need to do "sudo killall --wait /usr/bin/mongo
d".)
```

Capping Expensive Resources

There is another situation where the exit trap is very useful: if your script initiates an expensive resource, needed only while the script is executing, and you want to make certain it releases that resource once it's done. For

example, suppose you are working with Amazon Web Services (AWS), and want a script that creates a new image.

(If you're not familar with this: Servers running on the Amazon cloud are called "instances (http://aws.amazon.com/ec2/)". Instances are launched from Amazon Machine Images, a.k.a. "AMIs" or "images". AMIs are kind of like a snapshot of a server at a specific moment in time.)

A common pattern for creating custom AMIs looks like:

1. Run an instance (i.e. start a server) from some base AMI.
2. Make some modifications to it, perhaps by copying a script over and then executing it.
3. Create a new image from this now-modified instance.
4. Terminate the running instance, which you no longer need.

That last step is **really important**. If your script fails to terminate the instance, it will keep running and accruing charges to your account. (In the worst case, you won't notice until the end of the month, when your bill is way higher than you expect. Believe me, that's no fun!)

If our AMI-creation is encapsulated in a script, we can set an exit trap to destroy the instance. Let's rely on the EC2 command line tools:

```
#!/bin/bash
# define the base AMI ID somehow
ami=$1
# Store the temporary instance ID here
instance=''
# While we are at it, let me show you another use for a scratch
directory.
scratch=$(mktemp -d -t tmp.XXXXXXXXXX)
function finish {
    if [ -n "$instance" ]; then
        ec2-terminate-instances "$instance"
    fi
    rm -rf "$scratch"
}
trap finish EXIT
# This line runs the instance, and stores the program output (w
hich
# shows the instance ID) in a file in the scratch directory.
ec2-run-instances "$ami" > "$scratch/run-instance"
# Now extract the instance ID.
instance=$(grep '^INSTANCE' "$scratch/run-instance" | cut -f 2)
```

At this point in the script, the instance (EC2 server) is running [2]. You can do whatever you like: install software on the instance, modify its configuration programatically, et cetera, finally creating an image from the final version. The instance will be terminated for you when the script exits - even if some uncaught error causes it to exit early. (Just make sure to block until the image creation process finishes.)

Plenty Of Uses

I believe what I've covered in this article only scratches the surface; having used this bash pattern for years, I still find new interesting and fun ways to apply it. You will probably discover your own situations where it will help make your bash scripts more reliable.

Footnotes

1. The -t option to mktemp is optional on Linux, but needed on OS X. Make your scripts using this idiom more portable by including this option.
2. When getting the instance ID, instead of using the scratch file, we could just say:
   `instance=$(ec2-run-instances "$ami" | grep '^INSTANCE' | cut -f 2)`.
   But using the scratch file makes the code a bit more readable, leaves us with better logging for debugging, and makes it easy to capture other info from ec2-run-instances's output if we wish.